ATELIER 10 Le pont se lève

Algorithmie comparative - J4 - Jeudi 12 novembre 2026, 15h00 - 125 minutes, en binôme

La question à laquelle l'atelier répond

Le 8 septembre à 14h02, le pont Chaban-Delmas se lève : 23 tournées à recalculer, 11 minutes de recalcul, les tablettes figées (ticket T-0733). Comment recalculer les distances par la route, au mètre près, pendant qu'un livreur attend au feu ?

Ce que vous produisez : adr/GRILLE-ADR-007.md et adr/ADR-007-chemins.md (carnet), deux CSV dans bench/resultats.

Le rendu attendu

En fin d'atelier : le code, les mesures et la grille passent le correcteur, et l'ADR existe en premier jet (chaque rubrique en une à trois lignes, avec vos chiffres et le nom de vos CSV). Le correcteur sans KO avant 17h00, le commit du binôme à 17h00. Ensuite, l'ADR s'améliore chez vous jusqu'au dépôt du carnet, le 1er décembre.

Avant 17h00, le correcteur de l'atelier répond sans KO :

```
OK  À COMPLÉTER : Dijkstra avec tas rend les distances de Dijkstra simple
OK  Dijkstra avec tas ne remet pas dans le tas un carrefour déjà fixé
OK  À COMPLÉTER : A* rend les distances de Dijkstra
OK  L'heuristique guide vraiment A*
OK  Dijkstra simple est mesuré
OK  Dijkstra avec tas et A* sont mesurés jusqu'aux 23 tournées, et concordent
OK  La grille comparative de l'ADR-007 est remplie
OK  L'ADR-007 a ses six rubriques, remplies
OK  La rubrique 3 de l'ADR-007 renvoie à une mesure qui existe, la rubrique 5 donne le domaine de validité
OK  Les trois phrases de l'ADR-007 tiennent sans jargon
10 contrôles, 0 KO
```

Avant de commencer. Le tas de l'atelier 6 sert ici : votre Dijkstra et A* rangent les carrefours en attente dans bench/variantes/tas.ts. Si le premier contrôle du correcteur 6, le tas binaire, est KO : annexe ; ses autres KO ne gênent pas l'atelier 10. Si le jeu moyenne n'existe pas : annexe. Créez l'ADR-007 du binôme, que vous remplissez dès le sous-objectif 2 :

Windows	`Copy-Item adr\ADR-000-gabarit.md adr\ADR-007-chemins.md`
macOS et Linux	`cp adr/ADR-000-gabarit.md adr/ADR-007-chemins.md`

Pour chaque sous-objectif, la première commande et ce que vous devez voir ; les suivantes sont dans le README du dépôt (tableau des commandes de la journée 4) et dans les fiches atelier numéro 5 à 7, celles de mardi.

Les sous-objectifs

1. Compléter Dijkstra avec tas, lancer les tests consigne 1 de la slide

Pourquoi. Dijkstra sans tas relit tous les carrefours à chaque tour pour trouver le plus proche. Avec le tas, le plus proche sort tout seul, à condition que chaque voisin amélioré y soit entré : c'est la partie que vous écrivez.

Première action. Dans bench/variantes/chemins.ts, dijkstraSimple, écrite en entier, est votre modèle ; dans dijkstraTas, le commentaire À COMPLÉTER dit ce qui manque. Écrivez le relâchement, quatre lignes : pour chaque rue qui part du carrefour fixé, la distance du voisin en passant par lui ; si elle est meilleure que celle déjà notée, notez-la et mettez le voisin en attente dans le tas, avec cette distance pour priorité. Puis lancez `npm test`.

Windows	`code bench\variantes\chemins.ts`
macOS et Linux	`code bench/variantes/chemins.ts`

Vous devez voir. Le test de Dijkstra avec tas passe au vert. Avant vos quatre lignes, il échoue et affiche des -1 à la place des distances : un -1 veut dire que l'arrêt n'a jamais été atteint. Les deux autres tests des plus courts chemins passaient déjà.

```
✔ Dijkstra avec tas rend les distances de Dijkstra simple (455.566942ms)
```

2. Rejouer la levée du pont consigne 2 de la slide

Pourquoi. Voir le 8 septembre sur le réseau réel, avec les distances que votre Dijkstra vient de calculer.

Première action. Rejouez la levée du pont ; relevez en rubrique 1 de l'ADR-007 les tournées en cours, celles qui passaient par le pont et le détour.

Les trois systèmes	`npm run pont`

Vous devez voir. 23 tournées à recalculer : les 21 qui passaient par le pont et deux autres à vérifier, le chiffre du ticket ; les mêmes sur tous les postes, seuls les temps changent. La ligne astar reste lente jusqu'au sous-objectif 3. Si la commande s'arrête sur ARRÊT, revenez au sous-objectif 1 : le message compare vos distances aux distances attendues.

```
[pont] 75 tournées en cours à 14h02 (jeu moyenne), 21 passaient par le Pont Jacques Chaban-Delmas
[pont] tournées à recalculer : 23 (15 vélos cargo, 8 utilitaires)
[pont] reste à parcourir : 2465,7 km pont ouvert, 2562,2 km pont levé, soit 96,6 km de détour
```

3. Compléter la priorité de A*, relancer les tests consigne 3 de la slide

Pourquoi. La priorité de A*, distance parcourue plus vol d'oiseau, le guide vers l'arrêt : c'est le second À COMPLÉTER.

Première action. Dans aEtoile, h est donnée : le vol d'oiseau d'un carrefour à l'arrêt. Changez deux priorités, celle du départ et celle de chaque voisin amélioré : la distance déjà parcourue plus h. Lancez `npm test`, puis le correcteur.

Les trois systèmes	`npm run verifier -- 10`

Vous devez voir. Les trois tests des plus courts chemins passent, mais celui de A* passait déjà avant : seul le contrôle "L'heuristique guide vraiment A*" du correcteur dit si h entre dans la priorité. Ses quatre premiers contrôles sont alors OK.

4. Mesurer les trois méthodes consigne 4 de la slide

Pourquoi. Le critère du ticket est le temps des 23 tournées. Sans tas, une seule tournée prend déjà des secondes.

Première action. Mesurez Dijkstra sans tas sur une et deux tournées, puis le tas et A* sur 1, 5 et 23 tournées (suite dans le README).

Les trois systèmes	`npm run bench -- --sujet chemins --variantes simple,tas --tailles 1,2 --repetitions 3`

Vous devez voir. Des secondes sans tas, des dizaines de millisecondes avec, par tournée, et la même somme de contrôle : les distances sont identiques au mètre. Si le harness s'arrête sur ARRÊT, votre Dijkstra avec tas n'est pas juste : sous-objectif 1.

```
variante        taille   duree_ms   memoire_ko
simple               1    5053.54          939
tas                  1       8.85          127
```

5. Poser le travelling salesman problem consigne 5 de la slide

Pourquoi. Les distances sont justes ; l'ordre des arrêts, personne ne l'a recalculé. On ne cherche pas à le résoudre aujourd'hui.

Première action. Lancez sur votre poste l'essai de tous les ordres des prochains arrêts d'une tournée, de 6 à 11 arrêts.

Les trois systèmes	`npm run voyageur`

Vous devez voir. Le temps grandit à chaque arrêt ajouté ; la commande extrapole ensuite à 15 et 40 arrêts, au même rythme. Vos durées ne sont pas celles de la démo de 14h36 : le rythme change d'un poste à l'autre, jamais le nombre d'ordres.

```
[voyageur] 11 arrêts :  3,99 x 10^7 ordres essayés en   1618 ms, le meilleur fait 35,77 km
[voyageur] 15 arrêts : 1,31 x 10^12 ordres, soit 14,7 heures au même rythme
```

> **À préparer pour l'ADR**
> Combien d'ordres possibles pour les 15 arrêts qui restent à un livreur, combien de temps pour tous les essayer, et qu'en concluez-vous en rubrique 5 de l'ADR-007 ?

6. Remplir la grille, écrire l'ADR-007, vérifier consignes 6, 7 de la slide

Pourquoi. Les poids du ticket : temps 4, mémoire 1, lisibilité 2, robustesse 2, coût 1. Temps des 23 tournées : 5 sous 1 s, 4 sous 10 s, 3 sous 1 min, 2 sous 11 min (le 8 septembre), 1 au-delà. Les quatre autres barèmes sont en annexe.

Première action. Copiez la grille, notez les trois méthodes, puis écrivez l'ADR-007 : sa rubrique 5 dit à combien d'arrêts par livreur A* passerait devant (calcul en annexe). L'option A est Dijkstra sans tas ; le legacy du 8 septembre (11 minutes) se cite en rubrique 1, il ne se note pas. Préparez la décision et la question de compréhension à deux, puis relancez le correcteur.

Windows	`Copy-Item adr\grille-comparative.md adr\GRILLE-ADR-007.md`
macOS et Linux	`cp adr/grille-comparative.md adr/GRILLE-ADR-007.md`

- 125 minutes : l'ADR-007 s'écrit en entier ; ensuite, les KO des correcteurs 8 et 9 et la fin de vos ADR-005 et 006.
- Rubrique 3 : remplacez les colonnes du tableau du gabarit par 1, 2, 5 et 23 tournées, puis la mémoire.
- Rubrique 6 : ni unité (ms, Mo), ni nom de méthode ou de structure.

Vous devez voir. Le correcteur répond sans KO, comme en première page : Dijkstra, A*, le tas, les mesures, la grille, l'ADR.

> **La décision de l'atelier**
> Quelle méthode de recalcul retenez-vous quand le pont se lève, et pourquoi pas l'autre méthode rapide ?

> **La question de compréhension**
> À quoi sert ce que vous venez de faire, en deux phrases ? Préparez-la à deux : en fin d'atelier, l'intervenant passe la demander à quelques binômes.

À 17h00, le binôme commite : git add -A puis git commit -m "J4". Sans commit, le travail du jour n'est pas évalué.

Annexe, à consulter au besoin

Les cas particuliers

Le correcteur 10 renvoie au correcteur 6 : le tas de l'atelier 6 est faux. correction_J3.zip range le corrigé dans correction/J3 et ne remplace aucun de vos fichiers : faites le commit de votre travail, décompressez le zip à la racine du dépôt, copiez le tas corrigé à la place du vôtre, puis relancez le correcteur 6 :

Windows	`Expand-Archive -Path $HOME\Downloads\correction_J3.zip -DestinationPath . -Force`
	`Copy-Item correction\J3\tas.ts bench\variantes\tas.ts`
	`npm run verifier -- 6`

macOS et Linux	`unzip -o ~/Downloads/correction_J3.zip -d .`
	`cp correction/J3/tas.ts bench/variantes/tas.ts`
	`npm run verifier -- 6`

Vous êtes encore bloqués sur Dijkstra avec tas à 16h00 : correction_dijkstra.zip, publié sur Moodle à cette heure-là, remplace bench/variantes/chemins.ts par le fichier où le relâchement est écrit ; la priorité de A* y reste à compléter. Attention : un binôme qui a déjà changé les priorités de A* perd ce travail, ce zip ne concerne que les binômes bloqués au sous-objectif 1. Décompressez-le à la racine du dépôt, puis relancez les tests :

Windows	`Expand-Archive -Path $HOME\Downloads\correction_dijkstra.zip -DestinationPath . -Force` `npm test`
macOS et Linux	`unzip -o ~/Downloads/correction_dijkstra.zip -d .` `npm test`

npm run pont répond que le jeu moyenne est absent : générez-le, quelques secondes :

Les trois systèmes	`npm run data -- --echelle moyenne`

Les barèmes des critères sans chronomètre

- **Mémoire** (poids 1) : 5 sous 50 Mo, 4 sous 500 Mo, 3 au-delà. C'est la colonne memoire_ko du harness à 23 tournées, lue comme un ordre de grandeur ; deux options à moins d'un facteur 2 l'une de l'autre ont la même note. Dijkstra sans tas n'est mesuré qu'à une et deux tournées : sa mémoire n'y dépasse pas quelques Mo, ce qui suffit pour le noter.
- **Lisibilité** (poids 2) : lisez les trois méthodes de bench/variantes/chemins.ts, écrites avant vous sauf le relâchement de Dijkstra avec tas et la priorité de A*, et répondez pour chacune à trois questions : que fait la méthode, comment trouve-t-elle le prochain carrefour à fixer, où changer l'arrêt visé. Listez les idées qu'il faut connaître pour répondre ; on part de 5 et chaque idée (le tas et ses entrées périmées, l'heuristique et sa condition) coûte un point.
- **Robustesse** (poids 2) : la justesse au mètre et l'absence d'hypothèse à tenir. Barème : 5 si la méthode rend les mêmes distances que Dijkstra au mètre, sans hypothèse sur les données ; 3 si elle n'est juste que sous une hypothèse à tenir (une heuristique qui ne doit jamais surestimer, à revoir si l'unité change) ; 2 si elle rend des distances fausses en silence sur le scénario du pont ; 1 si `npm test` échoue.
- **Coût de mise en oeuvre** (poids 1) : les lignes à écrire et ce qu'il faut réutiliser et maintenir. Barème : 5 si rien n'est à écrire ; 4 pour quelques lignes sans structure à maintenir ; 3 pour quelques lignes et une structure du dépôt à réutiliser et à maintenir (le tas de l'atelier 6, le vol d'oiseau) ; 2 pour une structure nouvelle dans l'équipe ; 1 si une dépendance s'ajoute.
- Dans tous les cas, justifiez la note en une phrase sous la grille : le barème dit la note, la phrase dit ce que vous avez observé. Écrire le barème avant de mesurer n'interdit pas d'exécuter le code pour noter la robustesse : ce qui s'écrit avant, c'est la règle ; la mesure vient ensuite.
- **Le seuil de la rubrique 5** : divisez le temps du tas pour 23 tournées par 23 (une recherche par livreur), puis celui de A* par 23 x 15 (une recherche par arrêt, quinze arrêts au plus par livreur). Le rapport des deux est le nombre d'arrêts sous lequel A* passe devant.

Où trouver un exemple dans le code

bench/variantes/chemins.ts contient Dijkstra sans tas, écrit en entier : son relâchement est le modèle du vôtre. Les lignes 19 à 21 du même fichier disent ce qu'on met dans le tas : un carrefour et sa priorité. bench/graphe/graphe.ts contient volOiseau.

Si ça coince

- npm run pont ou le harness s'arrête sur "ARRÊT : dijkstraTas ne rend pas les distances de dijkstraSimple". Le message montre les deux séries de distances sur un réseau de six carrefours, décrit dans bench/variantes/chemins.ts : suivez votre boucle à la main sur ce réseau.
- Dijkstra avec tas trouve -1 partout : aucun voisin n'entre dans le tas. Le modèle note la distance ; vous devez aussi mettre le voisin en attente.
- Dijkstra avec tas trouve des distances plus longues que Dijkstra simple : une distance déjà notée est remplacée par une moins bonne. Relisez la condition du modèle.

- Le correcteur dit qu'un carrefour déjà fixé retourne dans le tas : un voisin y entre même quand sa distance ne s'améliore pas. Regardez ce qui dépend du si, et ce qui n'en dépend pas.
- Le correcteur 10 renvoie au correcteur 6 : le tas de l'atelier 6 est faux (annexe).
- Le test "A* rend les distances de Dijkstra simple" échoue : la priorité d'un voisin n'est pas sa distance déjà parcourue plus ce que h rend pour lui (h seul, ou h d'un autre carrefour).
- Le correcteur dit qu'A* fixe autant de carrefours que Dijkstra : l'heuristique n'entre pas dans la priorité des voisins.

Toujours bloqué après ces pistes : levez la main, ou écrivez sur Teams.

Si vous avez fini

- Mesurez Dijkstra sans tas sur deux tournées dans le scénario du pont, puis comparez l'extrapolation : `npm run pont -- --simple 2`.

Les sorties montrées sont de vraies sorties, capturées en jouant la fiche atelier en cours ; vos durées seront différentes, vos nombres de carrefours, d'adresses retrouvées, de tournées et vos distances non.